Politecnico di Milano

Ingegneria del Software – a.a. 2004/05

Appello del 15 luglio 2005

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ?, Ghezzi ?, Morzenti ?, SanPietro ?

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 28/30.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 10)

Il metodo statico *highLowNum* riceve in ingresso due array *nums* e *highs* e un intero *n* positivo. L'array nums, di lunghezza non precisata, contiene dei numeri interi, per ipotesi tutti diversi tra di loro; l'array *highs* è lungo esattamente *n*. Il metodo trova gli *n* numeri interi più grandi di *nums* e li inserisce in ordine decrescente nell'array *highs*; l'array *nums* non viene modificato.

a) Si scriva la specifica formale in JML del metodo.

```
//@ requires ..........................................................
..........................................................................
//@ ensures ...........................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
..........................................................................
```

```
public static void highLowNums (int [] nums, int [] highs, int n)
```

b) Si trasformi la specifica JML del punto precedente sostituendo una a scelta delle precondizioni nel lancio di opportune eccezioni.

Esercizio 2 (punti 10)

Si consideri la classe ***NumeroBinario*** che rappresenta numeri interi, positivi e senza segno in binario (ad esempio, 1110_2 corrisponde al numero 14_{10}). Si supponga che la classe fornisca:

- il metodo ***int dimensione()***, per conoscere il numero di cifre del ***NumeroBinario*** considerato;
- il costruttore ***NumeroBinario(int n)***, per creare un ***NumeroBinario*** di n bit inizializzati a 0;
- il metodo ***void cambiaBit(int pos, int val)***, per assegnare al bit di posizione pos il valore val;
- due iteratori per scorrere le cifre binarie dalla meno significativa alla più significativa (***Iterator destraSinistra()***) e dalla più significativa alla meno significativa (***Iterator sinistraDestra()***).

Utilizzando solamente i metodi elencati sopra, si realizzino i seguenti metodi:

2 – **`simmetrico`** per sapere se la sequenza di bit è simmetrica (palindroma):

```
public boolean simmetrico(){




}
```

3- **`specchio`** per invertire l'ordine di memorizzazione della sequenza di bit:

```
public NumeroBinario specchio(){




}
```

4 - Supponendo che la classe usi un Vector per memorizzare la sequenza di cifre binarie e un int per memorizzare il numero (per esempio se il Vector è 1110 il valore è 14), dichiarate come segue,

Vector bits;
int val;

si implementi il costruttore che prende in ingresso un numero intero e memorizza le cifre binarie nel Vector a partire da quella meno significativa:

```
public NumeroBinario(Integer n){

}
```

5 - Si scriva l'invariante di rappresentazione come private invariant in JML:

6 - Si realizzi anche l'iteratore per poter implementare il metodo ***Iterator destraSinistra()***

Esercizio 3 (punti 8)

Disegnare un diagramma delle classi UML che rappresenti una rete di computer. Questa si compone di nodi, i quali possono essere di 2 tipi: host e router. Gli host sono connessi ad esattamente un router, mentre i router possono essere connessi ad un numero qualunque di host e ad almeno un router. I nodi di una rete sono connessi tra loro mediante link fisici. Un link fisico può collegare più host e più router tra loro. Ogni connessione tra nodi della rete e link fisici è caratterizzata da un indirizzo di IP. Un host nella rete può offrire dei servizi. Ogni servizio, su un certo host, è caratterizzato da una porta. Inoltre ogni servizio si caratterizza per il tipo di protocollo su cui è trasportato, che può essere o TCP, o UDP.

Soluzioni

Esercizio 1

Parte A

```
//@ requires n>0 && nums != null && highs != null && highs.length==n && nums.length>=n &&
//@ (\forall int i; 0<=i && i<nums.length-1;
//@       (\forall int j; i<j && j<nums.length; nums[i] != nums[j]));
//@ ensures (\forall int i; 0<=i && i<nums.length; nums[i]==\old(nums[i])) &&
//@ (\forall int i; 0<=i && i<highs.length;
//@      (\exists int j; 0<j && j<nums.length; highs[i]==nums[j] ))  &&
//@ (\forall int i; 0<=i && i<highs.length-1; highs[i]>highs[i+1])  &&
//@ (\forall int i; 0<=i && i<highs.length;
//@         (\numof int j; 0<=j && j<nums.length; highs[i]<=nums[j]) <=n);
```

Parte B

```
//@ requires n>0 && highs.length==n && nums.length>=n &&
//@ (\forall int i; 0<=i && i<nums.length-1;
      (\forall int j; i<j && j<nums.length; nums[i] != numes[j]));
//@ ensures nums!=null && highs!=null && …poi come sopra
//@ signals (NullPointerException e)   (nums == null || highs == null);

public static void highLowNums (int [] nums, int [] highs, int n) throws NullPointerException
```

Esercizio 2

Parte 1

```
public boolean simmetrico(){
        Iterator iSD = this.sinistraDestra();
        Iterator iDS = this.destraSinistra();
        while (iSD.hasNext() && iDS.hasNext())
                if (((Integer) iSD.next()).intValue()
         != ((Integer) iDS.next()).intValue()) return false;
        return true;
}
```

NB: in questo modo la verifica della simmetria è effettuata due volte per ciascuna coppia di bit.

Parte 2

```
public NumeroBinario specchio(){
        NumeroBinario tmp = new NumeroBinario(this.dimensione());
        int n = 0;

        for (Iterator itr = this.destraSinistra(); itr.hasNext();)
                tmp.cambiaBit(n++, ((Integer) itr.next()).intValue());

        return tmp;
}
```

Parte 3

```
public NumeroBinario(Integer n){
        int tmp;

        val = n.intValue();
        bits = new Vector();

        tmp = val;
        while(tmp/2 != 0){
                bits.add(new Integer(tmp%2));
                tmp=tmp/2;
```

```
            }
            bits.add(new Integer(tmp%2));
                }
```

NB: si intende qui che bits contenga nella prima posizione la cifra piu' significativa (cosi' come specificato anche dall'invariante di rappresentazione). Per partire invece dalla cifra meno significativa si puo' chiamare opportunamente il metodo specchio() (modificando anche l'invariante).

Parte 4

```
//@ private invariant bits != null && bits. size()>0 &&
//@ (\forall int i; i>=0 && i<bits.size(); ((Integer) bits.get(i)).intValue() == 0
//@       || ((Integer) bits.get(i)).intValue() == 1) &&
//@ val == (\sum int i; i>=0 && i<bits.size();
//@           Math.pow(2, i)*((Integer) bits.get(i)).intValue());
```

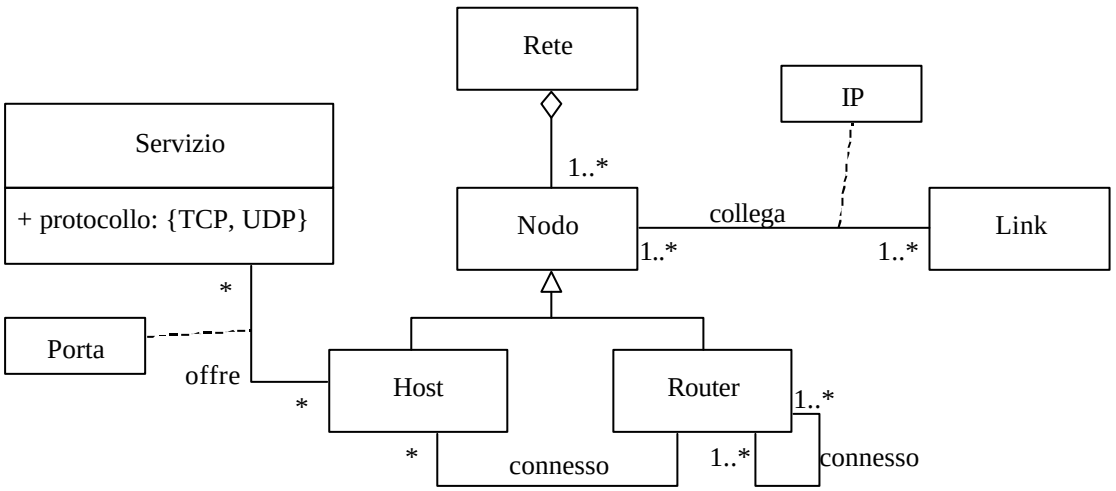
Parte 5

```
    private static class ItrDS implements Iterator {
            private NumeroBinario num;
            private int cp;

            ItrDS(NumeroBinario n){
                    num = n;
                    cp = n.dimensione();
            }

            public boolean hasNext(){
                    return cp>0;
            }

            public Object next() throw NoSuchElementException {
                    if (cp<=0) throw new NoSuchElementException();
                    cp--;
                    return num.bits.get(cp);
            }

    }
```

Esercizio 3